Recent Advancements in Natural Language Processing

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

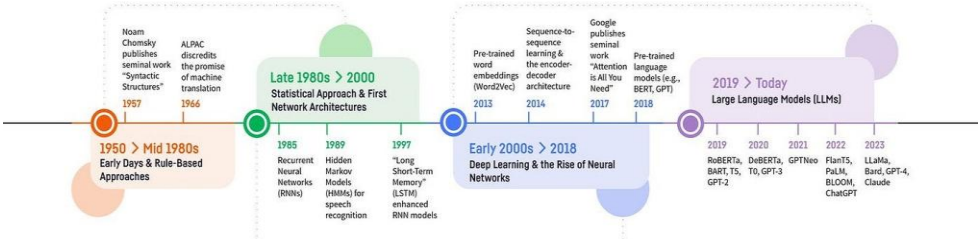
جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

Aug 16, 2026

The History of NLP

data iku



Source: Dataiku.

Table of Contents

Motivation

Four departures from the standard recipe of an autoregressive transformer over subword tokens:

- **IO-aware kernels:** FlashAttention keeps exact attention but reorganizes its arithmetic around the GPU memory hierarchy, so the $N \times N$ attention matrix never touches slow memory.
- **Open-weight LLMs:** Releases like the LLaMA 3 herd bring frontier-level models into the open, widening access to research and deployment.
- **Tokenizer-free modeling:** the Byte Latent Transformer (BLT) replaces subword tokenization with learned byte patches.
- **Non-autoregressive generation:** Large Language Diffusion Models (LLDMs) generate text by iterative denoising instead of left-to-right prediction.

Learning Outcomes

By the end of this session, you should be able to:

- Explain why attention is memory-bound rather than compute-bound, and how FlashAttention combines tiling, online softmax, and recomputation into exact IO-efficient attention.
- Describe the LLaMA 3 herd of models and what open-weight frontier models enable.
- Explain the Byte Latent Transformer (BLT): why tokenization is a bottleneck, and how entropy-based byte patching replaces it.
- Summarize Large Language Diffusion Models (LLDMs) and how diffusion-based text generation differs from autoregressive decoding.
- Situate these emerging paradigms relative to the standard transformer recipe and identify the trade-offs each one makes.

FlashAttention

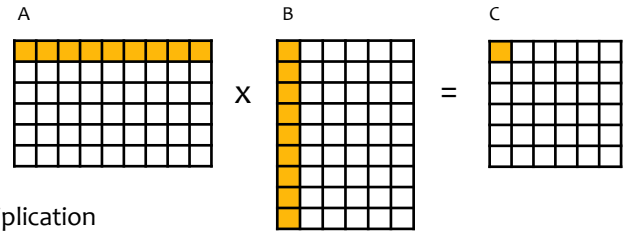
Scale Brings Quality and Capabilities

Core Challenge with Scale: Efficiency

Background

TILING FOR MATRIX MULTIPLICATION

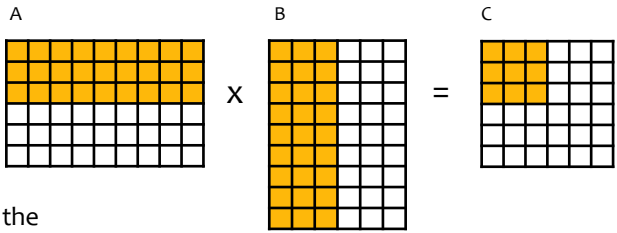
Tiling for Matrix Multiplication



- Matrix multiplication computes each output value as a dot-product of a row/column pair from the input matrices

$$C_{ij} = \sum_{m=1}^{M} \sum_{n=1}^{N} A_{im} B_{nj}$$

Tiling for Matrix Multiplication

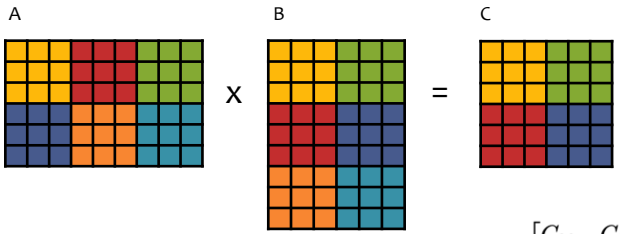


- We can view the computation as decomposing if we consider subsets of rows/columns

$$C_{(1:3,\ 1:3)} = A_{(1:3,\ 1:9)} \times B_{(1:9,\ 1:3)}$$

Tiling for Matrix Multiplication

- Tiling capitalizes on this decomposition
- Each output tile is computed by multiplying a pair of input tiles and adding it to the appropriate output tile



$$A = \begin{bmatrix} A_{00} & A_{01} & A_{02} \\ A_{10} & A_{11} & A_{12} \end{bmatrix}$$

with each $A_{ij} \in \mathbb{R}^{3\times3}$

$$B = \begin{bmatrix} B_{00} & B_{01} \\ B_{10} & B_{11} \\ B_{20} & B_{21} \end{bmatrix}$$

with each $B_{ij} \in \mathbb{R}^{3\times3}$

$$C = \begin{bmatrix} C_{00} & C_{01} \\ C_{10} & C_{11} \end{bmatrix}$$

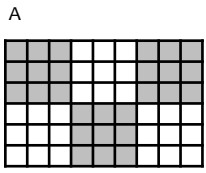
with each $C_{ij} \in \mathbb{R}^{3\times3}$

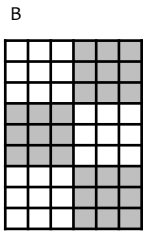
$$C_{00} = A_{00}B_{00} + A_{01}B_{10} + A_{02}B_{20}$$
$$C_{01} = A_{00}B_{01} + A_{01}B_{11} + A_{02}B_{21}$$
$$C_{10} = A_{10}B_{00} + A_{11}B_{10} + A_{12}B_{20}$$
$$C_{11} = A_{10}B_{01} + A_{11}B_{11} + A_{12}B_{21}$$

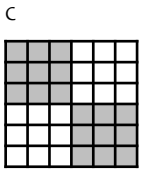
Tiling for Matrix Multiplication

- Tiling enables matrix multiplication of two **very** large matrices to capitalize on the small amount of fast memory on a device (e.g. GPU)
- Start by putting the input matrices and storage for the output matrix into large/slow memory
- Do the primary computation in slow/fast memory

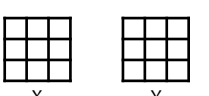
large/slow memory







$$C_{00} = A_{00}B_{00} + A_{01}B_{10} + A_{02}B_{20}$$



small/fast memory

$$X = A_{00}$$
$$Y = B_{00}$$
$$Z = XY$$
$$X = A_{01}$$
$$Y = B_{10}$$
$$Z = Z + XY$$

$$X = A_{02}$$
$$Y = B_{20}$$
$$Z = Z + XY$$

$$C_{00} = Z$$

Tiling for Self-Attention?

- It would be great if we could directly use tiling for self-attention
- Unfortunately, whereas the addition in matrix multiplication is associative, the softmax in self-attention is not!

$$\mathbf{X}' = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k})\mathbf{V}$$

Background

ONLINE SOFTMAX

Regular Softmax

- The standard softmax computation is used heavily throughout deep learning
- Yet, often we need to compute softmax on very large logits
- To avoid issues of overflow when raising e to some large power, we can use the safe softmax instead
- Every deep learning library implements this

$$y_i = \frac{e^{x_i}}{\sum_{j=1}^{V} e^{x_j}}$$

Algorithm 1 Naive softmax

1: $d_0 \leftarrow 0$
2: **for** $j \leftarrow 1, V$ **do**
3: $\quad d_j \leftarrow d_{j-1} + e^{x_j}$
4: **end for**
5: **for** $i \leftarrow 1, V$ **do**
6: $\quad y_i \leftarrow \frac{e^{x_i}}{d_V}$
7: **end for**

Figure from https://arxiv.org/pdf/1805.02867

Safe Softmax

- The standard softmax computation is used heavily throughout deep learning
- Yet, often we need to compute softmax on very large logits
- To avoid issues of overflow when raising e to some large power, we can use the safe softmax instead
- Every deep learning library implements this

$$y_i = \frac{e^{x_i - \max_{k=1}^{V} x_k}}{\sum_{j=1}^{V} e^{x_j - \max_{k=1}^{V} x_k}}$$

Algorithm 2 Safe softmax

$m_0 \leftarrow -\infty$
for $k \leftarrow 1, V$ **do**
    $m_k \leftarrow \max(m_{k-1}, x_k)$
end for
$d_0 \leftarrow 0$
for $j \leftarrow 1, V$ **do**
    $d_j \leftarrow d_{j-1} + e^{x_j - m_V}$
end for
for $i \leftarrow 1, V$ **do**
    $y_i \leftarrow \frac{e^{x_i - m_V}}{d_V}$
end for

Figure from https://arxiv.org/pdf/1805.02867

Online Softmax

- The problem with the usual safe softmax is that it requires three iterations, with each one accessing memory
- Online softmax reduces this to only two iterations through the data!
- This results in not only a 1.33x apparent speedup, but also a 1.3x speedup in practice because of reduced memory bandwidth requirements

Algorithm 3 Safe softmax with online normalizer calculation

$m_0 \leftarrow -\infty$
$d_0 \leftarrow 0$
for $j \leftarrow 1, V$ **do**
    $m_j \leftarrow \max\left(m_{j-1}, x_j\right)$
    $d_j \leftarrow d_{j-1} \times e^{m_{j-1}-m_j} + e^{x_j-m_j}$
end for
for $i \leftarrow 1, V$ **do**
    $y_i \leftarrow \frac{e^{x_i-m_V}}{d_V}$
end for

Figure from https://arxiv.org/pdf/1805.02867

Online Softmax

- The problem with the usual safe softmax is that it requires three iterations, with each one accessing memory
- Online softmax reduces this to only two iterations through the data!
- This results in not only a 1.33x apparent speedup, but also a 1.3x speedup in practice because of reduced memory bandwidth requirements

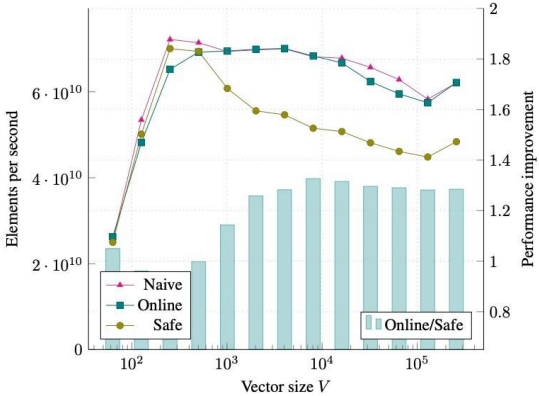


Figure 1: Benchmarking softmax, Tesla V100, fp32, batch size 4000 vectors

Figure from https://arxiv.org/pdf/1805.02867

FlashAttention

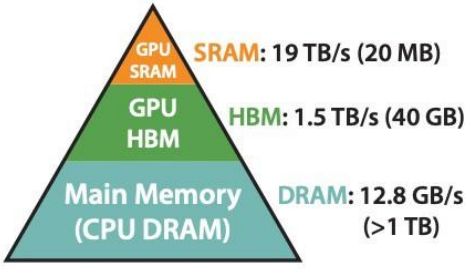
- One of the most impactful ideas in ML recently
- Even though many people probably don't even know they are using it!
- Introduced at HAET Workshop @ ICML July 2022
- Published @ NeurIPS Dec 2022

Figure from https://slideslive.com/38988230/flashattention-fast-and-memoryefficient-exact-attention-with-ioawareness

GPU Memory

Memory is arranged hierarchicaly

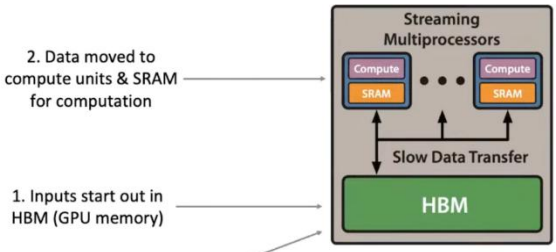
- GPU SRAM is small, and supports the fastest access
- GPU HBM is larger but with much slower access
- CPU DRAM is huge, but the slowest of all



Memory Hierarchy with Bandwidth & Memory Size

Figure from https://arxiv.org/pdf/2205.14135

GPU Memory


2. Data moved to compute units & SRAM for computation
Streaming Multiprocessors
Compute
SRAM
Compute
SRAM
Slow Data Transfer
1. Inputs start out in HBM (GPU memory)
HBM
3. Output written back to HBM


GPU SRAM
SRAM: 19 TB/s (20 MB)
GPU HBM
HBM: 1.5 TB/s (40 GB)

GPU Memory and Transformers

Transformer training is usually memory-bound

- Matrix multiplication takes up 99% of the FLOPS
- But only takes up 61% of the runtime
- Lots of time is wasted moving data around on the GPU
- Instead of doing computation

Table 1. Proportions for operator classes in PyTorch.

Operator class	% flop	% Runtime
△ Tensor contraction	99.80	61.0
□ Stat. normalization	0.17	25.5
○ Element-wise	0.03	13.5

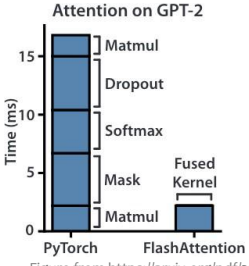
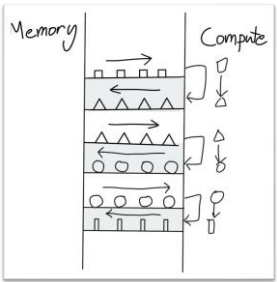


Figure from https://arxiv.org/pdf/2007.00072

Figure from https://arxiv.org/pdf/2205.14135

Operator Fusion

Version A: Usually, we compute a neural network one layer one at a time by moving the layer input to GPU SRAM (fast/small), doing some computation, then returning the output to GPU HBM (slow/large)



Version B: **Operator fusion** instead moves the original input to GPU SRAM (fast/small), does a whole sequence of layer computations without ever touching HBM, and then returns the final layer output to GPU HBM (slow/large)

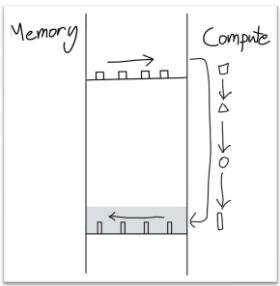


Figure from https://horace.io/brrr_intro.html

Operator Fusion

Version A: Usually, we compute a neural network one layer one at a time by moving the layer input to GPU SRAM (fast/small), doing some computation, then returning the output to GPU HBM (slow/large)

Version B: Operator fusion instead moves the original input to GPU SRAM (fast/small), does a whole sequence of layer computations without ever touching HBM, and then returns the final layer output to GPU HBM (slow/large)

Version A is exactly how standard attention is implemented

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

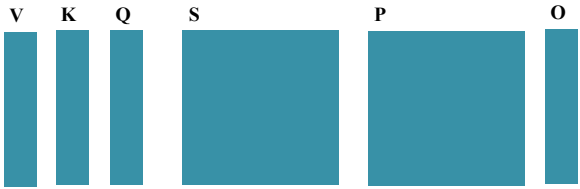
Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
4: Return $\mathbf{O}$.

Figure from https://arxiv.org/pdf/2205.14135

Standard Attention



Version A is exactly how standard attention is implemented

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \mathrm{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d},$$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top$, write $\mathbf{S}$ to HBM.
2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \mathrm{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{P}\mathbf{V}$, write $\mathbf{O}$ to HBM.
4: Return $\mathbf{O}$.

Figure from https://arxiv.org/pdf/2205.14135

FlashAttention

- Two key ideas are combined to obtain FlashAttention
- Both are well-established ideas, so the interesting part is how they are put together for attention
  1. **Tiling:** compute the attention weights block by block so that we don't have to load everything into SRAM at once
  2. **Recomputation:** don't ever store the full attention matrix, but just recompute the parts of it you need during the backward pass

FlashAttention: Tiling

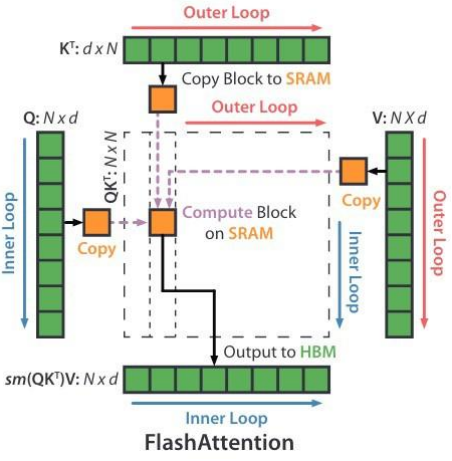


Figure from https://arxiv.org/pdf/2205.14135

FlashAttention uses tiling to prevent materialization of the large $N \times N$ attention matrix (dotted box) on (relatively) slow GPU HBM. In the outer loop (red arrows), FlashAttention loops through blocks of the K and V matrices and loads them to fast on-chip SRAM. In each block, FlashAttention loops over blocks of Q matrix (blue arrows), loading them to SRAM, and writing the output of the attention computation back to HBM

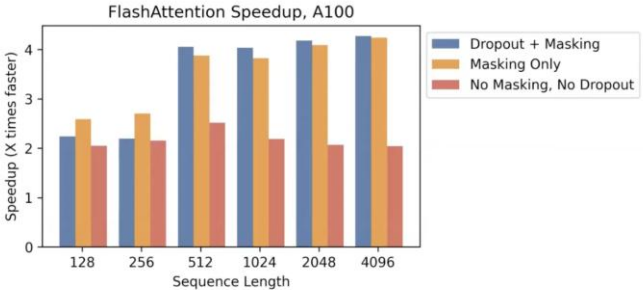
Algorithm 1 FlashAttention

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM, on-chip SRAM of size M.

1: Set block sizes $B_c = \lceil \frac{M}{4d} \rceil$, $B_r = \min\left(\lceil \frac{M}{4d} \rceil, d\right)$.
2: Initialize $\mathbf{O} = (0)_{N \times d} \in \mathbb{R}^{N \times d}$, $\ell = (0)_N \in \mathbb{R}^N$, $m = (-\infty)_N \in \mathbb{R}^N$ in HBM.
3: Divide $\mathbf{Q}$ into $T_r = \left\lceil \frac{N}{B_r} \right\rceil$ blocks $\mathbf{Q}_1, \ldots, \mathbf{Q}_{T_r}$ of size $B_r \times d$ each, and divide $\mathbf{K}, \mathbf{V}$ in to $T_c = \left\lceil \frac{N}{B_c} \right\rceil$ blocks $\mathbf{K}_1, \ldots, \mathbf{K}_{T_c}$ and $\mathbf{V}_1, \ldots, \mathbf{V}_{T_c}$, of size $B_c \times d$ each.
4: Divide $\mathbf{O}$ into T_r blocks $\mathbf{O}_i, \ldots, \mathbf{O}_{T_r}$ of size $B_r \times d$ each, divide ℓ into T_r blocks $\ell_i, \ldots, \ell_{T_r}$ of size B_r each, divide m into T_r blocks $m_1, \ldots, m_{T_r}$ of size B_r each.
5: **for** $1 \le j \le T_c$ **do**
6: Load $\mathbf{K}_j, \mathbf{V}_j$ from HBM to on-chip SRAM.
7: **for** $1 \le i \le T_r$ **do**
8: Load $\mathbf{Q}_i, \mathbf{O}_i, \ell_i, m_i$ from HBM to on-chip SRAM.
9: On chip, compute $\mathbf{S}_{ij} = \mathbf{Q}_i \mathbf{K}_j^T \in \mathbb{R}^{B_r \times B_c}$.
10: On chip, compute $\tilde{m}_{ij} = \text{rowmax}(\mathbf{S}_{ij}) \in \mathbb{R}^{B_r}$, $\tilde{\mathbf{P}}_{ij} = \exp(\mathbf{S}_{ij} - \tilde{m}_{ij}) \in \mathbb{R}^{B_r \times B_c}$ (pointwise), $\tilde{\ell}_{ij} = \text{rowsum}(\tilde{\mathbf{P}}_{ij}) \in \mathbb{R}^{B_r}$.
11: On chip, compute $m_i^{\text{new}} = \max(m_i, \tilde{m}_{ij}) \in \mathbb{R}^{B_r}$, $\ell_i^{\text{new}} = e^{m_i - m_i^{\text{new}}} \ell_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\ell}_{ij} \in \mathbb{R}^{B_r}$.
12: Write $\mathbf{O}_i \leftarrow \text{diag}(\ell_i^{\text{new}})^{-1} (\text{diag}(\ell_i) e^{m_i - m_i^{\text{new}}} \mathbf{O}_i + e^{\tilde{m}_{ij} - m_i^{\text{new}}} \tilde{\mathbf{P}}_{ij} \mathbf{V}_j)$ to HBM.
13: Write $\ell_i \leftarrow \ell_i^{\text{new}}$, $m_i \leftarrow m_i^{\text{new}}$ to HBM.
14: **end for**
15: **end for**
16: Return $\mathbf{O}$.

FlashAttention: Results

Benchmarking **runtime** against sequence length

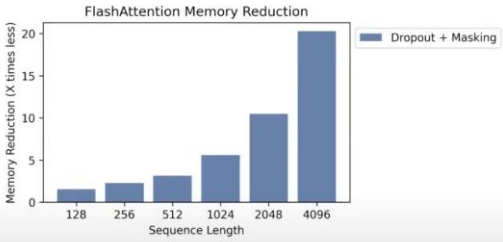


2-4x speedup over naive attention, esp. with dropout/masking

In paper: comparison with sparse/approximate attention

FlashAttention: Results

Benchmarking **memory footprint** against sequence length



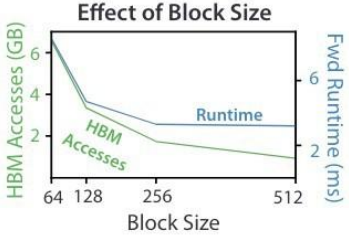
10-20x memory saving

Memory linear in sequence length — scaling up to 64K on one A100

FlashAttention: Results

- The algorithm is performing exact attention, so we see no reduction in perplexity or quality of the model
- The key metric is runtime

Attention	Standard	FLASHATTENTION
GFLOPs	66.6	75.2
HBM R/W (GB)	40.3	4.4
Runtime (ms)	41.7	7.3



FlashAttention: Results

- The algorithm is performing exact attention, so we see no reduction in perplexity or quality of the model
- The key metric is runtime

Model implementations	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Huggingface [87]	18.2	9.5 days (1.0×)
GPT-2 small - Megatron-LM [77]	18.2	4.7 days (2.0×)
GPT-2 small - FlashAttention	18.2	**2.7 days (3.5×)**
GPT-2 medium - Huggingface [87]	14.2	21.0 days (1.0×)
GPT-2 medium - Megatron-LM [77]	14.3	11.5 days (1.8×)
GPT-2 medium - FlashAttention	14.3	**6.9 days (3.0×)**

FlashAttention-2: Overview

- **Paper:** Dao et al. (2023)
- **Goal:** Memory-efficient and fast attention computation
- **Highlights:**
  - 90–95% GPU utilization
  - ~*8*× faster than standard attention
- **Techniques:**
  - Multi-query and grouped-query attention
  - Hardware-aware scheduling
  - Supports long context windows (32k+ tokens)
- **Adoption:**
  - Used in OpenAI GPT-4, Mistral, LLaMA 3
  - Critical for scaling to trillion-parameter models efficiently

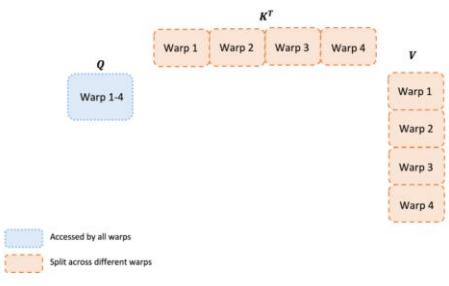
FlashAttention-2: Key Improvements

- **Fewer non-matmul FLOPs:**
  - Algorithm tweaks reduce non-matmul operations (e.g., rescaling, masking).
  - Maximizes use of specialized GPU units (Tensor Cores).
  - Non-matmul FLOPs are up to 16× more expensive than matmul FLOPs.
- **Better Parallelism:**
  - Parallelizes over sequence length in addition to batch size and heads.
  - Improves GPU utilization for long sequences and small batches.

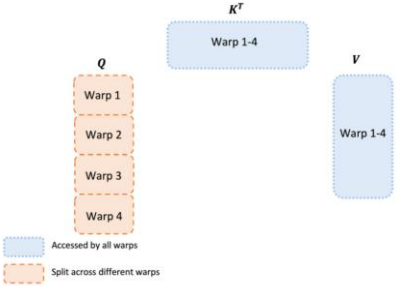
FlashAttention-2: Key Improvements (cont.)

- **Improved Work Partitioning:**
  - Splits Q across warps (instead of K/V), reducing shared memory communication.
  - Yields significant speedup by minimizing synchronization.
- **New Features:**
  - Supports head dimensions up to 256.
  - Enables multi-query and grouped-query attention.

FlashAttention-2: Architecture (cont.)

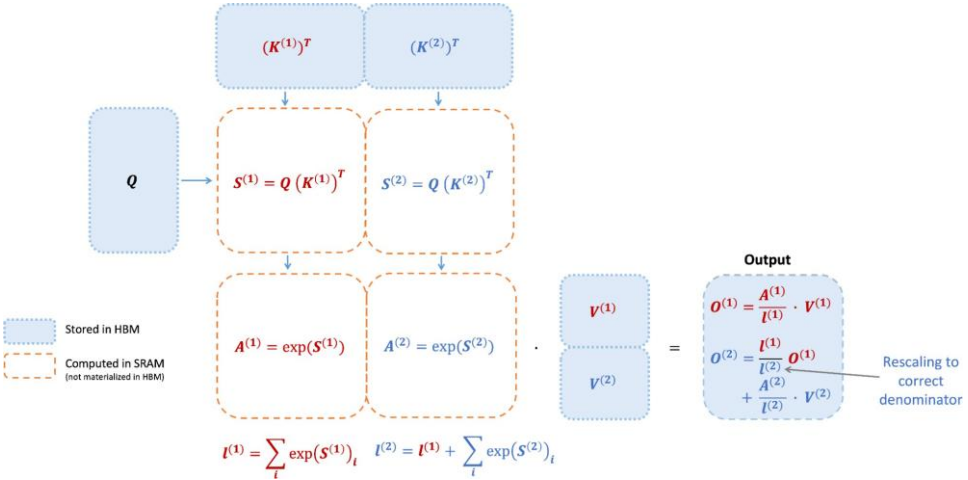


(a) FLASHATTENTION



(b) FLASHATTENTION-2

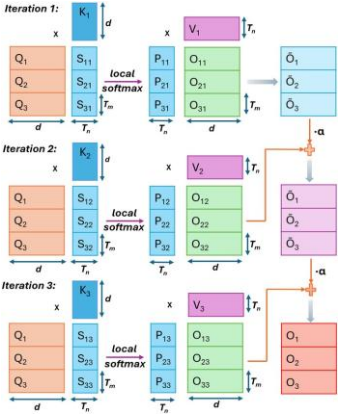
FlashAttention-2: Architecture (cont.)



FlashAttention-2: Architecture (cont.)

- **Multi-Query Attention:** Uses a single set of keys and values for all queries, reducing memory usage.
- **Grouped-Query Attention:** Groups queries to share keys and values, further optimizing memory.
- **Hardware-Aware Scheduling:** Optimizes GPU scheduling to maximize throughput and minimize latency.
- **Long Context Support:** Efficiently handles long sequences (32k+ tokens) with minimal memory overhead.

FlashAttention-2: Architecture (cont.)



Iterative update of output in FlashAttention-2 with Cn = 3 iterations.

FlashAttention-2: Performance Benchmarks

- **Speed:**
  - Up to 2× faster than FlashAttention-1, 9× faster than standard PyTorch attention.
  - Achieves up to 335 TFLOPs/s on H100 GPUs.
- **End-to-End Training:**
  - 1.3× speedup over optimized FlashAttention models.
  - Up to 225 TFLOPs/s on A100 GPU (72% model FLOPs utilization).
- **Example Results:**

Model	Baseline	FlashAttention	FlashAttention-2
GPT3-1.3B 2k	142	189	196
GPT3-1.3B 8k	72	170	220
GPT3-2.7B 2k	149	189	205
GPT3-2.7B 8k	80	175	225

- Baseline: Megatron-LM without FlashAttention.

Byte Latent Transformer (BLT)

Byte Latent Transformer (BLT)

Byte Latent Transformer: Patches Scale Better Than Tokens

Artidoro Pagnoni, **Ram Pasunuru**[†], **Pedro Rodriguez**[‡], **John Nguyen**[†], **Benjamin Muller**, **Margaret Li**[‡,◇], **Chunting Zhou**[◇], **Lili Yu**, **Jason Weston**, **Luke Zettlemoyer**, **Gargi Ghosh**, **Mike Lewis**, **Ari Holtzman**[†,2,◇], **Srinivasan Iyer**[†]

FAIR at Meta, [1]Paul G. Allen School of Computer Science & Engineering, University of Washington, [2]University of Chicago
[‡]Joint second author, [†]Joint last author, [◇]Work done at Meta

We introduce the Byte Latent Transformer (BLT), a new byte-level LLM architecture that, for the first time, matches tokenization-based LLM performance at scale with significant improvements in inference efficiency and robustness. BLT encodes bytes into dynamically sized patches, which serve as the primary units of computation. Patches are segmented based on the entropy of the next byte, allocating more compute and model capacity where increased data complexity demands it. We present the first FLOP controlled scaling study of byte-level models up to 8B parameters and 4T training bytes. Our results demonstrate the feasibility of scaling models trained on raw bytes without a fixed vocabulary. Both training and inference efficiency improve due to dynamically selecting long patches when data is predictable, along with qualitative improvements on reasoning and long tail generalization. Overall, for fixed inference costs, BLT shows significantly better scaling than tokenization-based models, by simultaneously growing both patch and model size.

Date: December 16, 2024
Correspondence: artidoro at cs.washington.edu, sviyer at meta.com
Code: https://github.com/facebookresearch/blt

Meta

Byte Latent Transformer (BLT) (cont.)

BLT is a compression-based approach for efficient LLMs:

- Compress input text into a smaller latent space
- Run transformer on latent tokens
- Decode with a decoder-only model

✓ Reduces memory & compute

✓ Maintains quality (BLEU, perplexity)

✓ Works well with long context (128k+)

Related: VQ-VAE, tokenizer-free models, compression transformers **Meta AI (2024)** https://ai.meta.com/research/publications/byte-latent-transformers-compression-based-efficiency-for-byte-level-llms/

Byte Latent Transformer (BLT) (cont.)

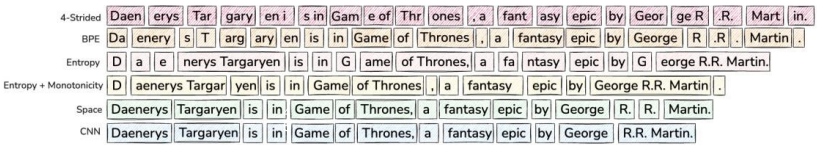


Figure 3 Patching schemes group bytes in different ways, each leading to a different number of resulting patches. Since each patch is processed using a large transformer step, the number of patches directly determines the bulk of the compute expended in terms of FLOPs. These schemes group bytes into patches by (a) striding every four bytes (§2.1) as in MegaByte (Yu et al., 2023), (b) tokenizing with Byte-Pair Encoding (BPE), in this case the Llama-3 (Dubey et al., 2024) tokenizer, (c & d) entropy-based patching as in this work (§2.3), (e) patching on space-bytes (Slagle, 2024), (f) and patching on entropy using a small CNN byte-level model with 2-byte context.

Byte Latent Transformer (BLT) (cont.)

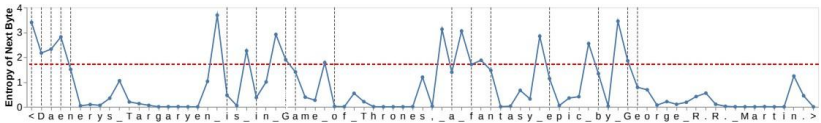


Figure 4 This figure plots the entropy $H(x_i)$ of each byte in "Daenerys Targeryen is in Game of Thrones, a fantasy epic by George R.R. Martin." with spaces shown as underscores. Patches end when $H(x_i)$ exceeds the global threshold θ_g, shown as a red horizontal line. The start of new patches are shown with vertical gray lines. For example, the entropies of "G" and "e" in "George R.R. Martin" exceed θ_g, so "G" is the start of a single byte patch and "e" of a larger patch extending to the end of the named entity as the entropy $H(x_i)$ stays low, resulting in no additional patches.

Byte Latent Transformer (BLT) (cont.)

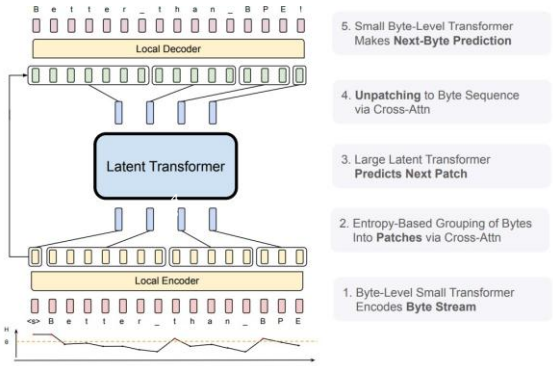


Figure 2 BLT comprises three modules, a lightweight *Local Encoder* that encodes input bytes into patch representations, a computationally expensive Latent Transformer over patch representations, and a lightweight *Local Decoder* to decode the next patch of bytes. BLT incorporates byte n-gram embeddings and a cross-attention mechanism to maximize information flow between the Latent Transformer and the byte-level modules (Figure 5). Unlike fixed-vocabulary tokenization, BLT dynamically groups bytes into patches preserving access to the byte-level information.

Encoder Multi-Headed Cross-Attention (cont.)

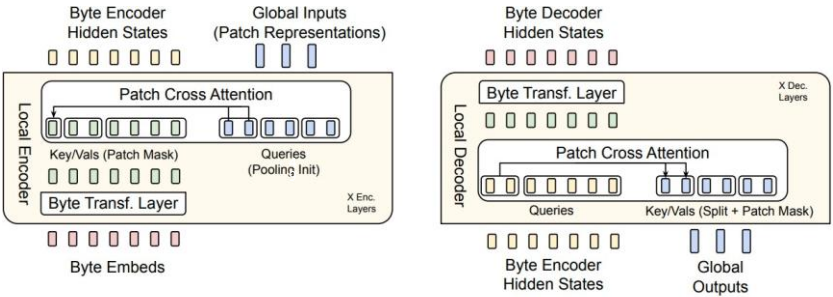


Figure 5 The local encoder uses a cross-attention block with patch representations as queries, and byte representations as keys/values to encode byte representations into patch representations. The local decoder uses a similar block but with the roles reversed i.e. byte representations are now the queries and patch representations are the keys/values. Here we use Cross-Attn $k = 2$.

BLT Performance

BLT Performance:

- BLT achieves state-of-the-art performance on long-context benchmarks.
- Outperforms existing LLMs in terms of perplexity and BLEU scores.
- Efficiently handles long sequences (128k+ tokens) with reduced memory and compute requirements.

BLT Performance

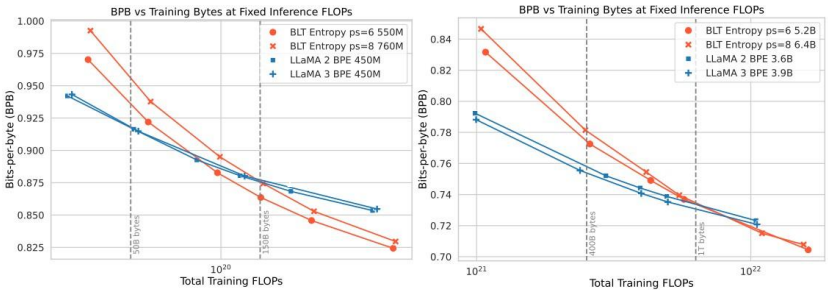


Figure 1 Scaling trends for fixed inference FLOP models (fully) trained with varying training budgets. In token-based models, a fixed inference budget determines the model size. In contrast, the BLT architecture provides a new scaling axis allowing simultaneous increases in model and patch size while keeping the same training and inference budget. BLT patch-size (ps) 6 and 8 models quickly overtake scaling trends of BPE Llama 2 and 3. Moving to the larger inference budget makes the larger patch size 8 model more desirable sooner. Both BPE compute-optimal point and crossover point are indicated with vertical lines.

BLT Performance (cont.)

Language	Language → English		English → Language	
	Llama 3	BLT	Llama 3	BLT
Arabic	22.3	24.6	10.4	8.8
German	41.3	42.0	29.8	31.2
Hindi	20.7	20.9	7.8	7.2
Italian	34.0	33.9	24.4	26.2
Vietnamese	31.2	31.0	28.4	23.7
Thai	17.9	18.1	10.5	7.7
Armenian	1.7	6.3	0.6	0.9
Amharic	1.3	3.1	0.4	0.5
Assamese	2.7	5.4	0.8	1.6
Bengali	4.7	12.7	1.7	4.1
Bosnian	36.0	37.3	16.9	19.6
Cebuano	18.2	20.6	5.8	9.1
Georgian	1.7	7.4	1.0	2.5
Gujarati	2.0	5.8	1.0	2.2
Hausa	5.75	5.9	1.2	1.3
Icelandic	16.1	17.9	4.8	5.3
Kannada	1.6	3.9	0.7	1.7
Kazakh	5.6	7.0	1.0	2.6
Kabuverdianu	20.3	20.9	5.1	6.8
Khmer	4.4	9.5	0.8	0.8
Kyrgyz	4.6	5.1	0.9	2.0
Malayalam	1.8	3.5	0.7	1.4
Odia	1.6	2.7	0.8	1.1
Somali	5.0	5.0	1.1	1.4
Swahili	10.1	12.0	1.4	2.3
Urdu	9.3	9.5	2.0	1.4
Zulu	4.7	5.0	0.6	0.5
Overall Average	12.1	**14.0**	5.9	**6.4**

Table 4 Performance of 8B BLT and 8B Llama 3 trained for 1T tokens on translating into and from six widely-used languages and twenty one lower resource languages with various scripts from the FLORES-101 benchmark (Goyal et al., 2022).

BLT Performance (cont.)

Task	Prompt	Llama 3	BLT
Substitute Word	Question: Substitute " and " with " internet " in " She went to the kitchen and saw two cereals. ". Answer:	She went to the kitchen and saw two cereals.	She went to the kitchen internet saw two cereals.
Swap Char	Question: Swap " h " and " a " in " that ". Answer:	that	taht
Substitute Char	Question: Substitutə " a " with " m " in " page ". Answer:	-	pmge
Semantic Similarity	Question: More semantically related to " are ": " seem ", " acre ". Answer:	acre	seem
Orthographic Similarity	Question: Closer in Levenshtein distance to " time ": " timber ", " period ". Answer:	period	timber
Insert Char	Question: Add an " z " after every " n " in " not ". Answer:	znotz	nzot

Figure 7 Output responses from Llama 3 and BLT models for various tasks from CUTE benchmark. BLT model performs better on sequence manipulation tasks compared to the tokenizer-based Llama 3 model. Note that few-shot examples are not shown in the above prompts to maintain clarity.

Large Language Diffusion Models (LLDMs)

LLDMs: Diffusion for Text Generation

Large Language Diffusion Models

Shen Nie [1 *†] Fengqi Zhu [1 *†] Zebin You [1†] Xiaolu Zhang [2‡] Jingyang Ou [1] Jun Hu [2‡] Jun Zhou [2]
Yankai Lin [1‡] Ji-Rong Wen [1] Chongxuan Li [1‡¶]

Abstract

Autoregressive models (ARMs) are widely regarded as the cornerstone of large language models (LLMs). We challenge this notion by introducing **LLaDA**, a diffusion model trained from scratch under the pre-training and supervised fine-tuning (SFT) paradigm. LLaDA models distributions through a forward data masking process and a reverse process, parameterized by a vanilla Transformer to predict masked tokens. By optimizing a likelihood bound, it provides a principled generative approach for probabilistic inference. Across extensive benchmarks, LLaDA demonstrates strong *scalability*, outperforming our self-constructed ARM baselines. Remarkably, LLaDA 8B is competitive with strong LLMs like LLaMA3 8B in *in-context learning* and, after SFT, exhibits impressive *instruction-following* abilities in case studies such as multi-turn dialogue. Moreover, LLaDA addresses the reversal curse, surpassing GPT-4o in a reversal poem completion task. Our findings establish diffusion models as a viable and promising alternative to ARMs, challenging the assumption that key LLM capabilities discussed above are inherently tied to ARMs. Project page and codes: https://ml-gsai.github.io/LLaDA-demo/.

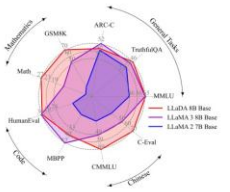


Figure 1. **Zero/Few-Shot Benchmarks.** We scale LLaDA to an unprecedented size of 8B parameters from scratch, achieving competitive performance with strong LLMs (Dubey et al., 2024).

distribution $p_{\text{data}}(\cdot)$ by optimizing a model distribution $p_\theta(\cdot)$ through maximum likelihood estimation, or equivalently KL divergence minimization between the two distributions:

$$\underbrace{\max_\theta \mathbb{E}_{p_{\text{data}}(x)} \log p_\theta(x) \Leftrightarrow \min_\theta \mathrm{KL}(p_{\text{data}}(x) || p_\theta(x))}_{\text{Generative modeling principles}}. \quad (1)$$

LLDMs: Diffusion for Text Generation (cont.)

- **Emerging trend:** Use diffusion instead of autoregression for text generation

Key Idea:

- Generate latent text representation over diffusion steps
- Use decoder to map latent $\rightarrow$ text

Benefits:

- Better global coherence
- Parallel generation
- Less repetition, hallucination

LLDMs: Diffusion for Text Generation (cont.)

Related work:

- Diffusion-LM (Li et al.)
- Masked-Diffusion Transformers
- UniDiffuser (Vision + Text)

Note: Still experimental, but promising for the future of text generation

LLDMs: Diffusion for Text Generation (cont.)

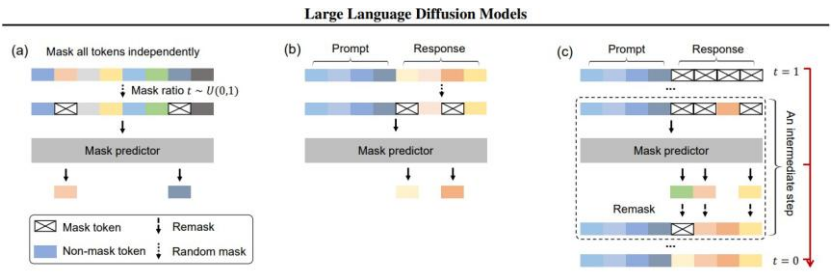


Figure 2. **A Conceptual Overview of LLaDA.** (a) Pre-training. LLaDA is trained on text with random masks applied independently to all tokens at the same ratio $t \sim U[0, 1]$. (b) SFT. Only response tokens are possibly masked. (c) Sampling. LLaDA simulates a diffusion process from $t = 1$ (fully masked) to $t = 0$ (unmasked), predicting all masks simultaneously at each step with flexible remask strategies.

LLDMs: Diffusion for Text Generation (cont.)

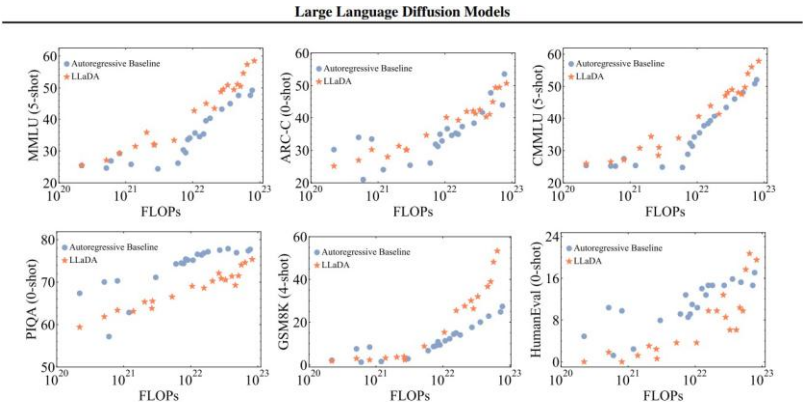


Figure 3. **Scalability of LLaDA.** We evaluate the performance of LLaDA and our ARM baselines trained on the same data across increasing computational FLOPs. LLaDA exhibits strong scalability, matching the overall performance of ARMs on six tasks.

LLDMs: Diffusion for Text Generation (cont.)

Table 1. **Benchmark Results of Pre-trained LLMs.** * indicates that LLaDA 8B Base, LLaMA2 7B Base, and LLaMA3 8B Base are evaluated under the same protocol, detailed in Appendix B.5. Results indicated by † and ¶ are sourced from Chu et al. (2024); Yang et al. (2024) and Bi et al. (2024) respectively. The numbers in parentheses represent the number of shots used for evaluation. "-" indicates unknown data.

	LLaDA 8B*	LLaMA3 8B*	LLaMA2 7B*	Qwen2 7B†	Qwen2.5 7B†	Mistral 7B†	Deepseek 7B¶
Model	Diffusion	AR	AR	AR	AR	AR	AR
Training tokens	2.3T	15T	2T	7T	18T	-	2T
General Tasks							
MMLU	**65.9** (5)	65.4 (5)	45.9 (5)	70.3 (5)	74.2 (5)	64.2 (5)	48.2 (5)
BBH	49.8 (3)	**57.6** (3)	37.3 (3)	62.3 (3)	70.4 (3)	56.1 (3)	39.5 (3)
ARC-C	47.9 (0)	**53.1** (0)	46.3 (0)	60.6 (25)	63.7 (25)	60.0 (25)	48.1 (0)
Hellaswag	72.5 (0)	**79.1** (0)	76.0 (0)	80.7 (10)	80.2 (10)	83.3 (10)	75.4 (0)
TruthfulQA	**46.4** (0)	44.0 (0)	39.0 (0)	54.2 (0)	56.4 (0)	42.2 (0)	-
WinoGrande	74.8 (5)	**77.3** (5)	72.5 (5)	77.0 (5)	75.9 (5)	78.4 (5)	70.5 (0)
PIQA	74.4 (0)	**80.6** (0)	79.1 (0)	-	-	-	79.2 (0)
Mathematics & Science							
GSM8K	**70.7** (4)	53.1 (4)	14.3 (4)	80.2 (4)	85.4 (4)	36.2 (4)	17.4 (8)
Math	**27.3** (4)	15.1 (4)	3.2 (4)	43.5 (4)	49.8 (4)	10.2 (4)	6.0 (4)
GPQA	**26.1** (5)	25.9 (5)	25.7 (5)	30.8 (5)	36.4 (5)	24.7 (5)	-
Code							
HumanEval	33.5 (0)	**34.2** (0)	12.8 (0)	51.2 (0)	57.9 (0)	29.3 (0)	26.2 (0)
HumanEval-FIM	**73.8** (2)	73.3 (2)	26.9 (2)	-	-	-	-
MBPP	38.2 (4)	**47.4** (4)	18.4 (4)	64.2 (0)	74.9 (0)	51.1 (0)	39.0 (3)
Chinese							
CMMLU	**69.9** (5)	50.7 (5)	32.5 (5)	83.9 (5)	-	-	47.2 (5)
C-Eval	**70.5** (5)	51.7 (5)	34.0 (5)	83.2 (5)	-	-	45.0 (5)

LLDMs: Diffusion for Text Generation (cont.)

Table 4. **Visualization of the Sampling Process and a Generated Multi-round Dialogue.** In the response of LLaDA, darker colors indicate tokens predicted in the later stages of sampling, while lighter colors correspond to earlier predictions.

Sampling Process	
User	Lily can run 12 kilometers per hour for 4 hours. After that, she runs 6 kilometers per hour. How many kilometers can she run in 8 hours?
LLaDA	In 4 hours , Lily runs 12 * 4 = 48 kilometers . After 4 hours she runs 6 * 4 = 24 kilometers . In total she runs 48 + 24 = 72 kilometers in 8 hours . Result 72
Multi-round Dialogue Case	
User	Could you please share with me what the initial two lines of the renowned poem 'The Road Not Taken'?
LLaDA	Certainly! The initial two lines of the renowned poem "The Road Not Taken" by Robert Frost are: "Two roads diverged in a yellow wood, And sorry I could not travel both." These lines set the stage for the poem, introducing the idea of a speaker facing a choice between two paths.
User	Please help me translate into Chinese.
LLaDA	Sure! The first two lines of "The Road Not Taken" by Robert Frost can be translated into Chinese as: "两条路分岔在黄色的树林中，遗憾我不能同时走"
User	Please translate into German.
LLaDA	Sure! The first two lines of "The Road Not Taken" by Robert Frost can be translated into German as: "Zwei Wege trennten sich im gelben Wald, und leider konnte ich nicht beide tun."
User	Please also write a poem about life choices. I require 5 sentences, and the first word of each sentence starts with C as the first letter.
LLaDA	Certainly! Here's a poem about life choices, with each sentence starting with the letter "C": Choices cascade through life's journey, Creating connections with each decision, Careful consideration leads to better paths, Courageous choices shape our destiny, Choosing wisely guides us through the years.

LLDMs: Algorithm Overview

Algorithm 1 Pre-training of LLaDA

Require: mask predictor p_θ, data distribution p_{data}

repeat
  $x_0 \sim p_{\text{data}}$, $t \sim \text{U}(0, 1]$ # with a probability of 1%, the sequence length of x_0 follows $\text{U}[1, 4096]$
  $x_t \sim q_{t|0}(x_t|x_0)$ # $q_{t|0}$ is defined in Eq. (7)
  Calculate $\mathcal{L} = -\frac{1}{t*L}\sum_{i=1}^{L} \mathbf{1}[x_t^i = \text{M}] \log p_\theta(x_0^i|x_t)$ # L is the sequence length of x_0
  Calculate $\nabla_\theta \mathcal{L}$ and run optimizer.
until Converged
Return p_θ

Pre-training. In 1% of steps the sequence length is drawn from *U*[1, 4096] instead of the full 4096, so the model also sees short inputs. Source: Nie et al., NeurIPS 2025, arXiv:2502.09992v2, Algorithm 1.

LLDMs: Algorithm Overview (cont.)

Algorithm 2 Supervised Fine-Tuning of LLaDA

Require: mask predictor p_θ, pair data distribution p_{data}

repeat
  $p_0, r_0 \sim p_{\text{data}}, t \sim \text{U}(0, 1]$ # please refer to Appendix B.1 for details on the SFT data processing.
  $r_t \sim q_{t|0}(r_t|r_0)$ # $q_{t|0}$ is defined in Eq. (7)
  Calculate $\mathcal{L} = -\frac{1}{t*L'} \sum_{i=1}^{L'} \mathbf{1}[r_t^i = \text{M}] \log p_\theta(r_0^i|p_0, r_t)$ # L' is the sequence length of r_0
  Calculate $\nabla_\theta \mathcal{L}$ and run optimizer.
until Converged
Return p_θ

Fine-tuning. Only the response r_0 is masked; the prompt p_0 is always visible, so the same loss now trains a conditional model.
Source: Nie et al., NeurIPS 2025, arXiv:2502.09992v2, Algorithm 2.

LLDMs: Algorithm Overview (cont.)

Algorithm 3 Conditional Log-likelihood Evaluation of LLaDA

Require: mask predictor p_θ, prompt p_0, response r_0, the number of Monte Carlo estimations n_{mc}

log_likelihood $= 0$
for $i \leftarrow 1$ to n_{mc} **do**
  $l \sim \{1, 2, \ldots, L\}$ # L is the sequence length of r_0
  Obtain r_l by uniformly sampling l tokens from r_0 without replacement for masking
  log_likelihood $=$ log_likelihood $+ \frac{L}{l} \sum_{i=1}^{L} \mathbf{1}[r_l^i = \mathrm{M}] \log p_\theta(r_0^i | p_0, r_l)$
end for
log_likelihood $=$ log_likelihood$/n_{mc}$
Return log_likelihood

Evaluation. There is no chain rule to multiply out, so likelihood is estimated by Monte Carlo: mask *l* random tokens, score them, and average over draws. Source: Nie et al., NeurIPS 2025, arXiv:2502.09992v2, Algorithm 3.

Summary

Comparative Overview

Advancement	What it removes	What it costs
FlashAttention	The $N \times N$ attention matrix in slow GPU memory	Extra FLOPs recomputing attention in the backward pass, and kernels rewritten per GPU generation
LLaMA 3	Closed weights at the frontier	The 405B model does not fit on one 8-GPU machine in BF16
BLT	The fixed subword vocabulary	A separately trained entropy model, and patch size as a new knob
LLaDA	Left-to-right decoding	No prefix to cache, and the answer length is fixed before sampling

Future Directions

Open problems the four papers leave behind:

- Attention kernels co-designed with the hardware, rather than rewritten by hand for every GPU generation.
- Scaling laws derived for byte-level models, instead of borrowing the compute-optimal schedule of subword transformers.
- Patch boundaries learned end to end, rather than handed down by a separate entropy model.
- Byte-ifying existing tokenizer-based checkpoints, so a byte-level model need not be trained from scratch.
- Serving machinery for bidirectional models: caching without a prefix, and generation lengths the model chooses itself.
- RL-based alignment for diffusion language models, which so far stop at supervised fine-tuning.

Key Takeaways

- **FlashAttention**: Exact attention got fast by cutting memory traffic, not arithmetic — tiling and online softmax in the forward pass, recomputation in the backward.
- **LLaMA 3**: Open weights at frontier quality widen access to research and deployment.
- **BLT**: Learned byte patches remove the tokenizer, trading fixed vocabulary for dynamic, entropy-driven compute allocation.
- **LLDMs**: Diffusion-based generation is an alternative to left-to-right decoding, enabling parallel refinement of whole sequences.
- Each advancement removes a different fixed assumption of the standard transformer recipe — one of them inside the kernel, three at the modeling level.

References

References

Papers covered:

- **FlashAttention**: Dao et al., NeurIPS 2022, *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness* (arXiv:2205.14135)
- **Online softmax**: Milakov & Gimelshein, 2018, *Online normalizer calculation for softmax* (arXiv:1805.02867)
- **Data movement**: Ivanov et al., MLSys 2021, *Data Movement Is All You Need: A Case Study on Optimizing Transformers* (arXiv:2007.00072)
- **LLaMA 3**: Llama Team, AI @Meta, 2024, *The Llama 3 Herd of Models* (arXiv:2407.21783)
- **BLT**: Pagnoni et al., 2024, *Byte Latent Transformer: Patches Scale Better Than Tokens* (arXiv:2412.09871)
- **LLaDA**: Nie et al., NeurIPS 2025, *Large Language Diffusion Models* (arXiv:2502.09992)

References (cont.)

Patching schemes and architectures:

- **MEGABYTE**: Yu et al., 2023, *Predicting Million-byte Sequences with Multiscale Transformers* (arXiv:2305.07185)
- **SpaceByte**: Slagle, 2024, *Towards Deleting Tokenization from Large Language Modeling* (arXiv:2404.14408)
- **Dynamic token pooling**: Nawrot et al., ACL 2023, *Efficient Transformers with Dynamic Token Pooling* (arXiv:2211.09761)
- **Perceiver**: Jaegle et al., ICML 2021, *General Perception with Iterative Attention* (arXiv:2103.03206)
- **Diffusion-LM**: Li et al., NeurIPS 2022, *Diffusion-LM Improves Controllable Text Generation* (arXiv:2205.14217)

References (cont.)

Covered in other lectures:

- **FlashAttention-2:** Dao, 2023, *Faster Attention with Better Parallelism and Work Partitioning* (arXiv:2307.08691) — see *Transformers: 2017 vs 2026*

References (cont.)

Benchmarks:

- **CUTE**: Edman, Schmid & Fraser, EMNLP 2024, *Measuring LLMs' Understanding of Their Tokens* (arXiv:2409.15452)
- **FLORES-101**: Goyal et al., *TACL* 10, 2022, pp. 522–538

Code and resources:

- FlashAttention kernels: https://github.com/Dao-AILab/flash-attention
- Horace He, *Making Deep Learning Go Brrrr From First Principles* (operator fusion): https://horace.io/brrr_intro.html
- BLT: https://github.com/facebookresearch/blt
- LLaDA: https://ml-gsai.github.io/LLaDA-demo/
- Open LLM Leaderboard: https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard